

Python - Higher Order Functions

Higher-order functions in Python allows you to manipulate functions for increasing the flexibility and re-usability of your code. You can create higher-order functions using nested scopes or callable objects.

Additionally, the **functools** module provides utilities for working with higher-order functions, making it easier to create decorators and other function-manipulating constructs. This tutorial will explore the concept of higher-order functions in Python and demonstrate how to create them.

What is a Higher-Order Function?

A higher-order function is a function that either, takes one or more functions as arguments or returns a function as its result. Below you can observe the some of the properties of the higher-order function in Python –

- A function can be stored in a variable.
- A function can be passed as a parameter to another function.
- A high order functions can be stored in the form of lists, hash tables, etc.
- Function can be returned from a function.

To create higher-order function in Python you can use nested scopes or callable objects. Below we will discuss about them briefly.

Creating Higher Order Function with Nested Scopes

One way to defining a higher-order function in Python is by using nested scopes. This involves defining a function within another function and returns the inner function.

Example

Let's observe following example for creating a higher order function in Python. In this example, the **multiplier** function takes one argument, *a*, and returns another function *multiply*, which calculates the value $a * b$

```
def multiplier(a):
    # Nested function with second number
    def multiply(b):
        # Multiplication of two numbers
        return a * b
    return multiply

# Assigning nested multiply function to a variable
multiply_second_number = multiplier(5)
# Using variable as high order function
Result = multiply_second_number(10)
# Printing result
print("Multiplication of Two numbers is: ", Result)
```

Output

On executing the above program, you will get the following results –

```
Multiplication of Two numbers is:  50
```

Creating Higher-Order Functions with Callable Objects

Another approach to create higher-order functions is by using callable objects. This involves defining a class with a **__call__** method.

Example

Here is the another approach to creating higher-order functions is using callable objects.

```

class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return self.factor * x

# Create an instance of the Multiplier class
multiply_second_number = Multiplier(2)

# Call the Multiplier object to computes factor * x
Result = multiply_second_number(100)

# Printing result
print("Multiplication of Two numbers is: ", Result)

```

Output

On executing the above program, you will get the following results –

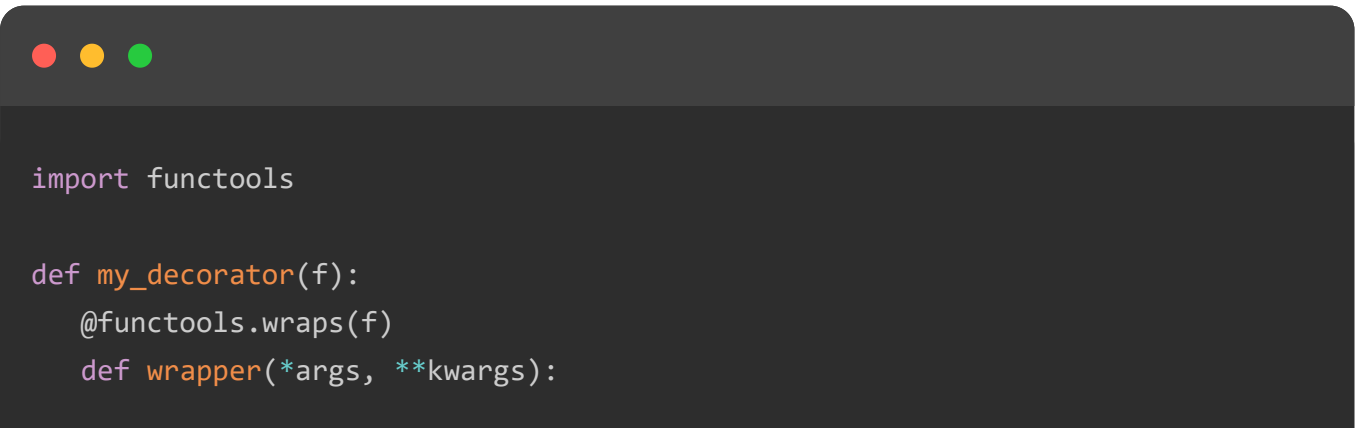
```
Multiplication of Two numbers is: 200
```

Higher-order functions with the 'functools' Module

The **functools** module provides higher-order functions that act on or return other functions. Any callable object can be treated as a function for the purposes of this module.

Working with Higher-order functions using the wraps()

In this example, **my_decorator** is a higher-order function that modifies the behavior of **invite** function using the **functools.wraps()** function.



```

import functools

def my_decorator(f):
    @functools.wraps(f)
    def wrapper(*args, **kwargs):

```

```

        print("Calling", f.__name__)
        return f(*args, **kwargs)
    return wrapper

@my_decorator
def invite(name):
    print(f"Welcome to, {name}!")

invite("Tutorialspoint")

```

Output

On executing the above program, you will get the following results –

```

Calling invite
Welcome to, Tutorialspoint!

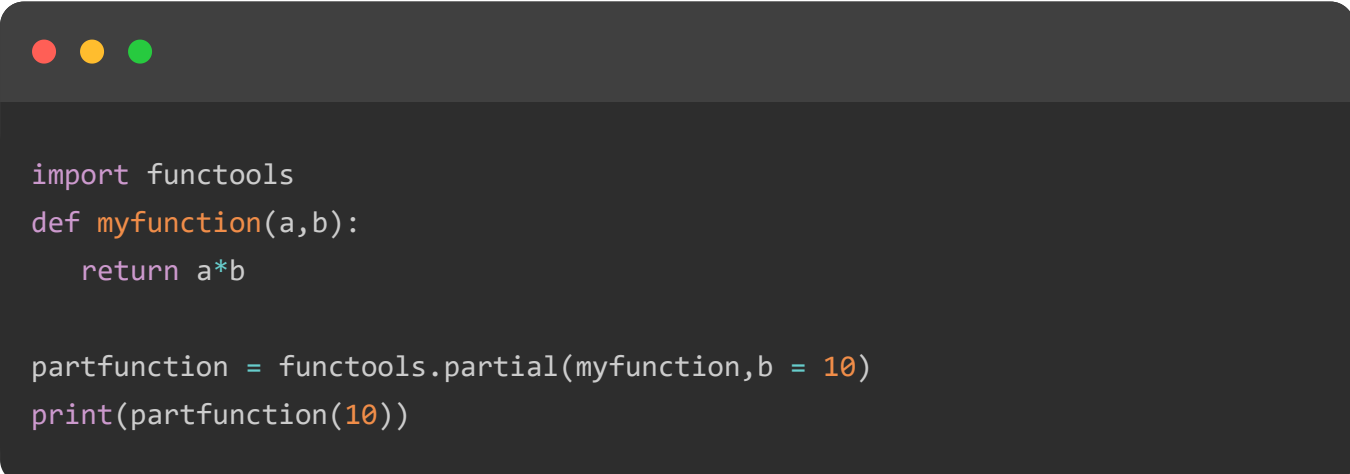
```

Working with Higher-order functions using the partial()

The **partial()** function of the **functools** module is used to create a callable 'partial' object. This object itself behaves like a function. The **partial()** function receives another function as argument and freezes some portion of a functions arguments resulting in a new object with a simplified signature.

Example

In following example, a user defined function **myfunction()** is used as argument to a partial function by setting default value on one of the arguments of original function.



```

import functools
def myfunction(a,b):
    return a*b

partfunction = functools.partial(myfunction,b = 10)
print(partfunction(10))

```

Output

On executing the above program, you will get the following results –

100

Working with Higher-order functions using the reduce()

Similar to the above approach the **functools** module provides the **reduce()** function, that receives two arguments, a function and an iterable. And, it returns a single value. The argument function is applied cumulatively two arguments in the list from left to right. Result of the function in first call becomes first argument and third item in list becomes second. This is repeated till list is exhausted.

Example

```
import functools
def mult(x,y):
    return x*y

# Define a number to calculate factorial
n = 4
num = functools.reduce(mult, range(1, n+1))
print (f'Factorial of {n}: ',num)
```

Output

On executing the above program, you will get the following results –

```
Factorial of 4:  24
```

TOP TUTORIALS

[Python Tutorial](#)[Java Tutorial](#)[C++ Tutorial](#)[C Programming Tutorial](#)[C# Tutorial](#)[PHP Tutorial](#)

[R Tutorial](#)
[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)
[Amazon Web Services Tutorial](#)
[Microsoft Azure Tutorial](#)
[Git Tutorial](#)
[Ethical Hacking Tutorial](#)
[Docker Tutorial](#)
[Kubernetes Tutorial](#)
[DSA Tutorial](#)
[Spring Boot Tutorial](#)
[SDLC Tutorial](#)
[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)
[Java & Spring Boot Advanced Certification](#)



Chapters ▾

Categories ≡

[Advanced Certification In Business Analytics](#)
[Artificial Intelligence And Machine Learning](#)
[DevOps Certification](#)
[Game Development Certification](#)
[Front-End Developer Certification](#)
[AWS Certification Training](#)
[Python Programming Certification](#)

COMPILERS & EDITORS

[Online Java Compiler](#)
[Online Python Compiler](#)
[Online Go Compiler](#)

[Online C Compiler](#)
[Online C++ Compiler](#)
[Online C# Compiler](#)
[Online PHP Compiler](#)
[Online MATLAB Compiler](#)
[Online Bash Terminal](#)
[Online SQL Compiler](#)
[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.